CPSC 375 High-Performance Computing

Lecture 2

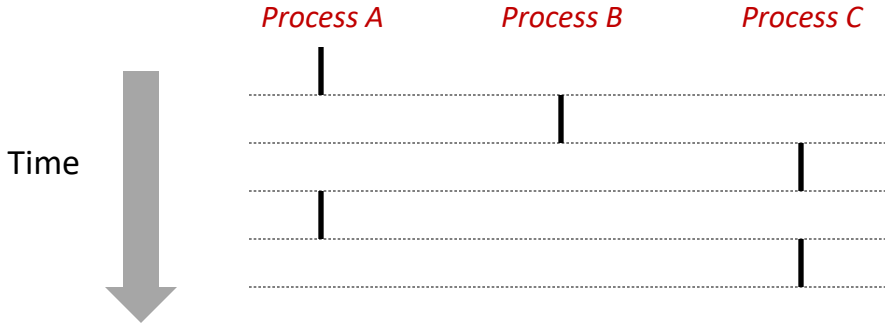
Processes

Process Concept

- Definition: A *process* is an instance of a running program.
  - One of the most profound ideas in computer science
  - Not the same as "program" or "processor"
- Process provides each program with two key abstractions:
  - Logical control flow
    - Each program seems to have exclusive use of the CPU
  - Private *virtual* address space
    - Each program seems to have exclusive use of main memory
- How are these Illusions maintained?
  - Process executions interleaved or run on separate cores
  - Address spaces managed by virtual memory system

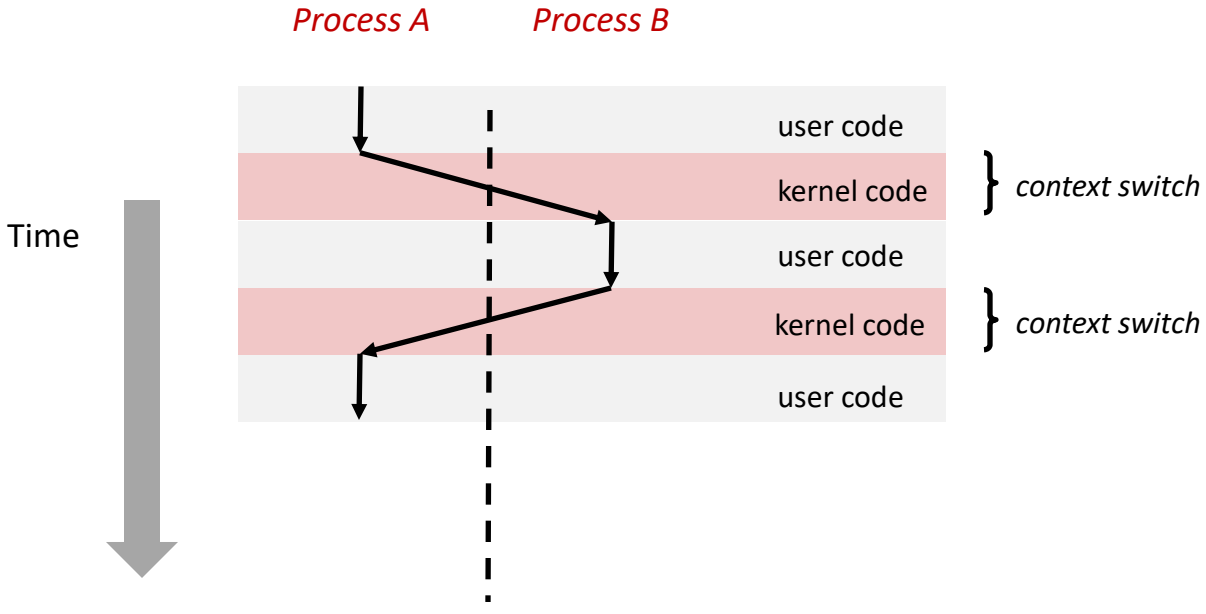
Concurrent Processes

- Two processes *run concurrently* if their flows overlap in time
- Otherwise, they are *sequential*
- Examples (running on single core):
  - Concurrent: A & B, A & C
  - Sequential: B & C



Context Switching

- Control flow passes from one process to another via a *context switch*



Creating New Processes

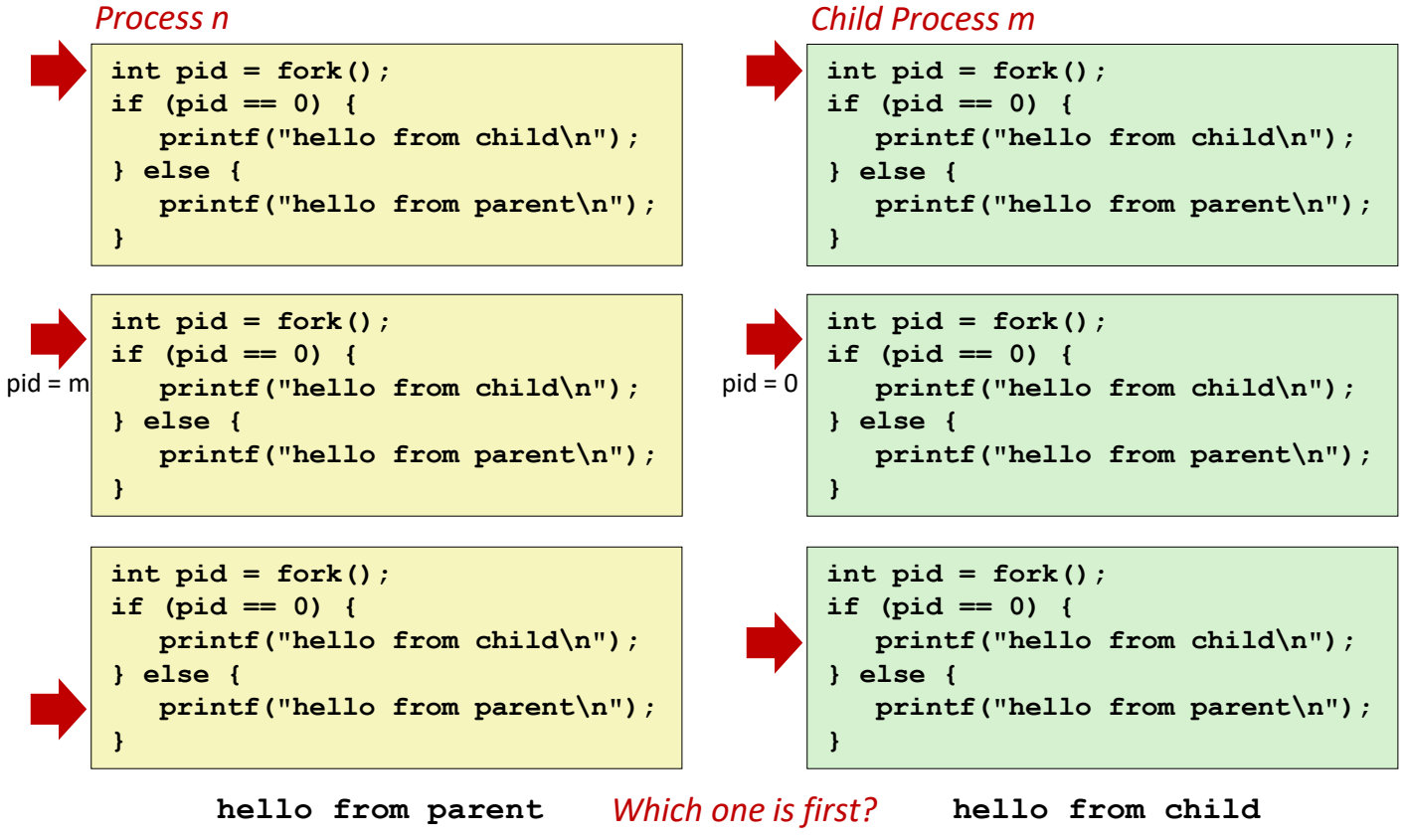
`int fork(void)`

- creates a new process (*child* process) that is identical to the calling process (*parent* process)

```
int pid = fork();
if (pid == 0)
   printf("hello from child\n");
else
   printf("hello from parent\n");
```

- called *once* but returns *twice*
  - returns 0 to the child process
  - returns child's **`pid`** to the parent process

Understanding `fork()`



`fork` Example

- Parent and child both run same code
  - Distinguish parent from child by return value from **`fork`**
- Start with same state, but each has private copy
  - Including shared output file descriptor
  - Relative ordering of their print statements undefined

```
void fork1() {
   int x = 1;
   int pid = fork();
   if (pid == 0)
      printf("Child has x = %d\n", ++x);
   else
      printf("Parent has x = %d\n", --x);
   printf("Bye from process %d with x = %d\n", getpid(), x);
}
```

Ending a process

`void exit(int status)`

- exits a process
- normally return with status 0

Zombies

- Idea
  - When process terminates, still consumes system resources
    - Various tables maintained by OS
  - Called a "zombie"
    - Living corpse, half alive and half dead
- Reaping
  - Performed by parent on terminated child
  - Parent is given exit status information
  - Kernel discards process
- What if parent doesn't reap?
  - If any parent terminates without reaping a child, then child will be reaped by `init` process

Zombie Example

```
void fork7()
{
    if (fork() == 0) {
        /* Child */
        printf("Terminating Child, PID = %d\n",
            getpid());
        exit(0);
    } else {
        printf("Running Parent, PID = %d\n",
            getpid());
        while (1)
            ; /* Infinite loop */
    }
}
```

```
linux> ./fork7 &
[1] 6639
Running Parent, PID = 6639
Terminating Child, PID = 6640
linux> ps
  PID TTY          TIME CMD
 6585 ttyp9    00:00:00 tcsh
 6639 ttyp9    00:00:03 fork7
 6640 ttyp9    00:00:00 fork7 <defunct>
 6641 ttyp9    00:00:00 ps
linux> kill 6639
[1]    Terminated
linux> ps
  PID TTY          TIME CMD
 6585 ttyp9    00:00:00 tcsh
 6642 ttyp9    00:00:00 ps
```

- `ps` shows child process as "defunct"
- Killing parent allows child to be reaped by `init`

Nonterminating Child Example

```
linux> ./fork8
Terminating Parent, PID = 6675
Running Child, PID = 6676
linux> ps
  PID TTY          TIME CMD
 6585 ttyp9    00:00:00 tcsh
 6676 ttyp9    00:00:06 fork8
 6677 ttyp9    00:00:00 ps
linux> kill 6676
linux> ps
  PID TTY          TIME CMD
 6585 ttyp9    00:00:00 tcsh
 6678 ttyp9    00:00:00 ps
```

```
void fork8()
{
    if (fork() == 0) {
        /* Child */
        printf("Running Child, PID = %d\n",
               getpid());
        while (1)
            ; /* Infinite loop */
    } else {
        printf("Terminating Parent, PID = %d\n",
               getpid());
        exit(0);
    }
}
```

- Child process still active even though parent has terminated
- Must kill explicitly, or else will keep running indefinitely

Synchronizing with child processes

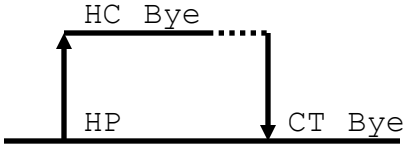
```
int wait(int *child_status)
```

- suspends current process until one of its children terminates
- return value is the `pid` of the child process that terminated
- if `child_status != NULL`, then the object it points to will be set to a status indicating why the child process terminated

Synchronizing with child processes, cont'd

```
void fork9() {
   int child_status;

   if (fork() == 0) {
      printf("HC: hello from child\n");
   }
   else {
      printf("HP: hello from parent\n");
      wait(&child_status);
      printf("CT: child has terminated\n");
   }
   printf("Bye\n");
   exit();
}
```



Checking Exit Status of Children

- If multiple children completed, will take in arbitrary order
- Can use macros WIFEXITED and WEXITSTATUS to get information about exit status

```
void fork10()
{
    int pid[N];
    int i;
    int child_status;
    for (i = 0; i < N; i++)
        if ((pid[i] = fork()) == 0)
            exit(100+i); /* child */
    for (i = 0; i < N; i++) {
        int wpid = wait(&child_status);
        if (WIFEXITED(child_status))
            printf("Child %d terminated with exit status %d\n",
                    wpid, WEXITSTATUS(child_status));
        else
            printf("Child %d terminate abnormally\n", wpid);
    }
}
```

Waiting for a Specific Process

`waitpid(pid, &status, options)`

- suspends current process until specific process terminates
- various options (see manpage of waitpid())

```
void fork11()
{
    int pid[N];
    int i;
    int child_status;
    for (i = 0; i < N; i++)
        if ((pid[i] = fork()) == 0)
            exit(100+i); /* Child */
    for (i = N-1; i >= 0; i--) {
        int wpid = waitpid(pid[i], &child_status, 0);
        if (WIFEXITED(child_status))
            printf("Child %d terminated with exit status %d\n",
                    wpid, WEXITSTATUS(child_status));
        else
            printf("Child %d terminated abnormally\n", wpid);
    }
}
```

Other useful system functions

`sleep(n)`

- suspends current process for `n` seconds.

`exec()`

- Family of functions to load and run a new program in the context of the current process. Does not return (unless error)
- Overwrites code, data, and stack
- keeps pid, open files and signal context

execve: Loading and Running Programs

```
pid = fork();
switch(pid)
{
 case -1:
    exit(1);

 case 0:
    execl("./child", "child", (char *) 0);
    exit(1); /* should never get here */

 default:
    printf("Process[%d]: parent in execution ...\n", getpid());
    sleep(5);
    if (wait(NULL) > 0) /* waiting for child */
       printf("Process[%d]: parent terminating child ...\n", getpid());
    printf("Process[%d]: parent terminating ...\n", getpid());
    exit(0);
}
```

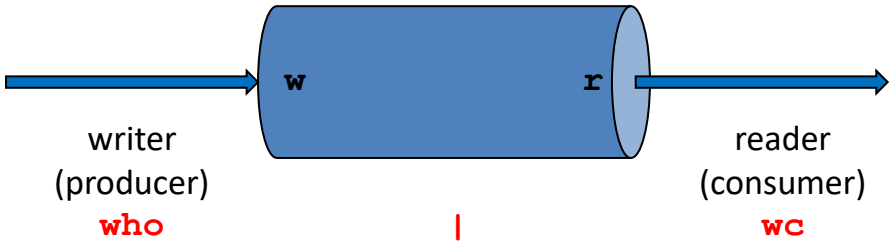
Interprocess Communication (IPC)

Linux IPC

- Pipes
- Message Queues
- Shared memory
- Semaphores
- Signals
- Sockets

Linux Pipes

- *pipe*: a circular buffer of fixed size written by one process and read by another



- OS enforces *mutual exclusion*: only one process at a time can access the pipe.

Creating Pipes

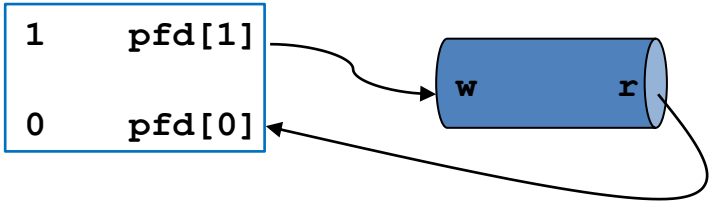
```
int pipe(int pipefd)
```

where

```
int pipefd[2];
```

- `pipe()` creates a pipe and returns two file descriptors,
  - `pipefd[0]` for reading
  - `pipefd[1]` for writing

Pipe Example




```
main() {
    int n;
    int pfd[2];
    char buff[100];

    if (pipe(pfd) < 0)          // create a pipe
         perror("pipe error");

    printf("readfd = %d, writefd = %d\n", pfd[0], pfd[1]);
```

Result:

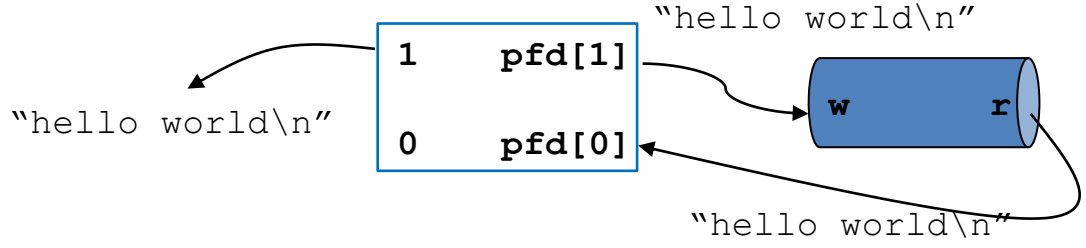
```
readfd = 3, writefd = 4
```

Pipe Example, cont'd

```
    if (write(pfd[1], "hello world\n", 12) != 12) // write to pipe
        perror("write error");

    if ((n=read(pfd[0], buff, sizeof(buff))) <= 0) // read from pipe
        perror("read error");

    write(STDOUT_FILENO, buff, n);    /* write to stdout */
    close(pipefd[0]);
    close(pipefd[1]);
    exit(0);
}
```



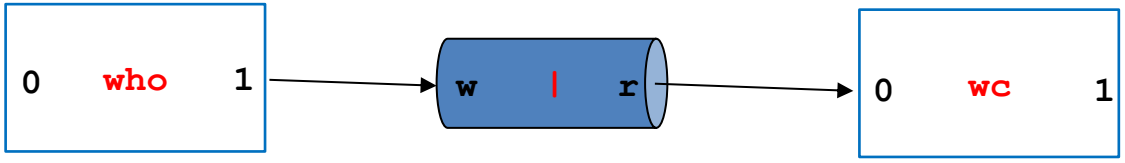
Result:

```
hello world
```

But how do you connect two processes with a pipe?

Connecting Processes Using Pipes

who | wc



Connecting Processes Using Pipes, cont'd

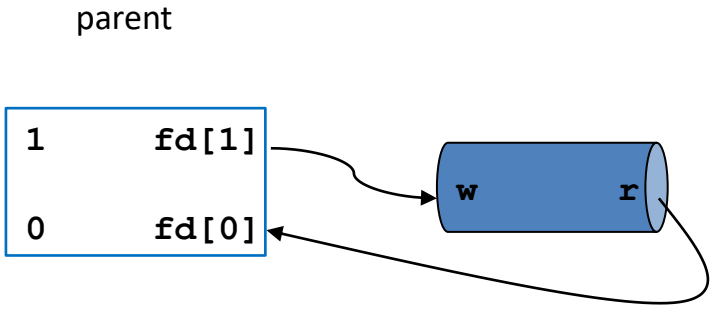
- Create new processes
- Each process running different parts of the same program
- Each process running different programs
- Putting it together with a pipe

whowc.c: parent process starts

parent

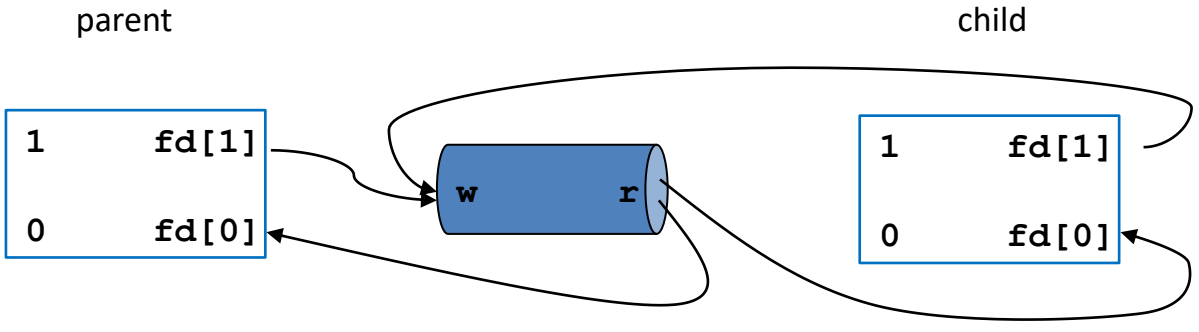
Parent Process

```
pipe(fd);
```



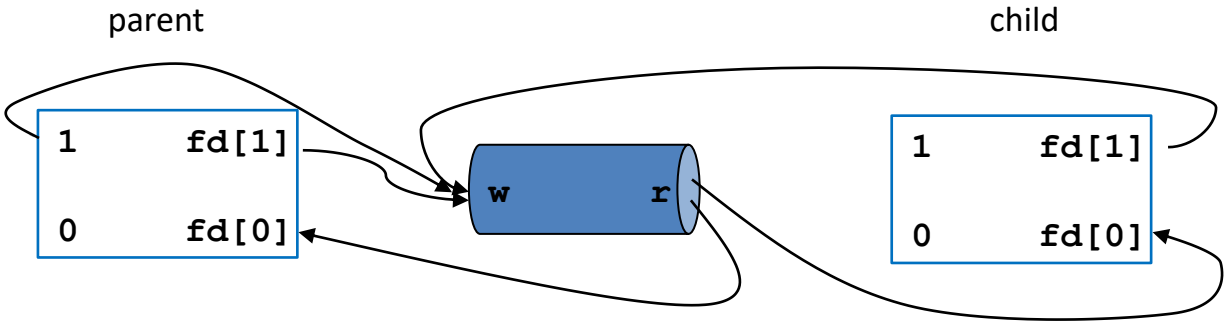
Parent Process

```
fork();
```



Parent Process

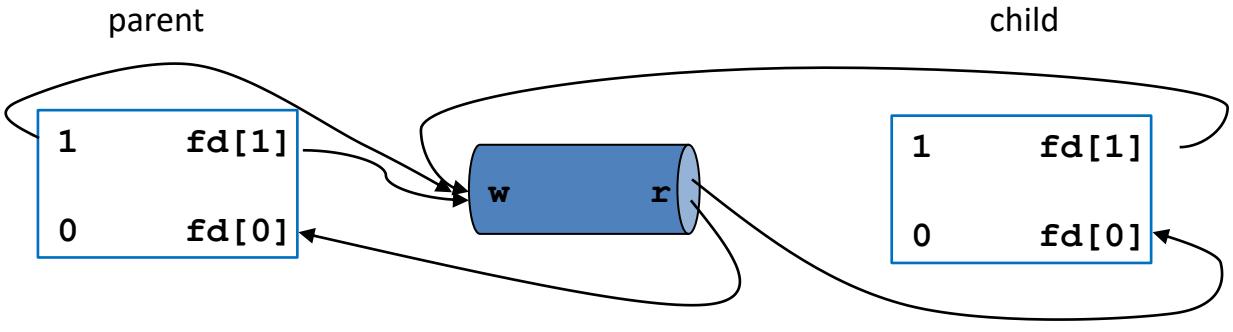
```
dup2(fd[1], STDOUT_FILENO);
```



`dup2(oldfd, newfd)` **makes** `newfd` **refer to the same file description as** `oldfd`.

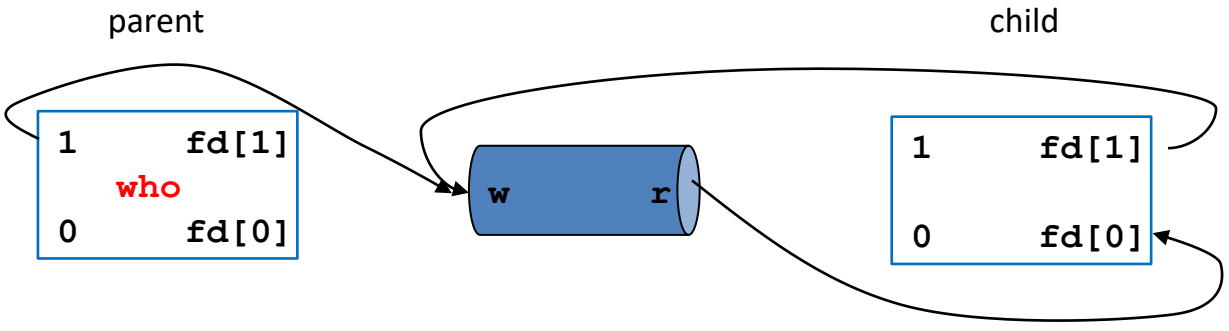
Parent Process

```
close(fd[0]);
close(fd[1]);
```



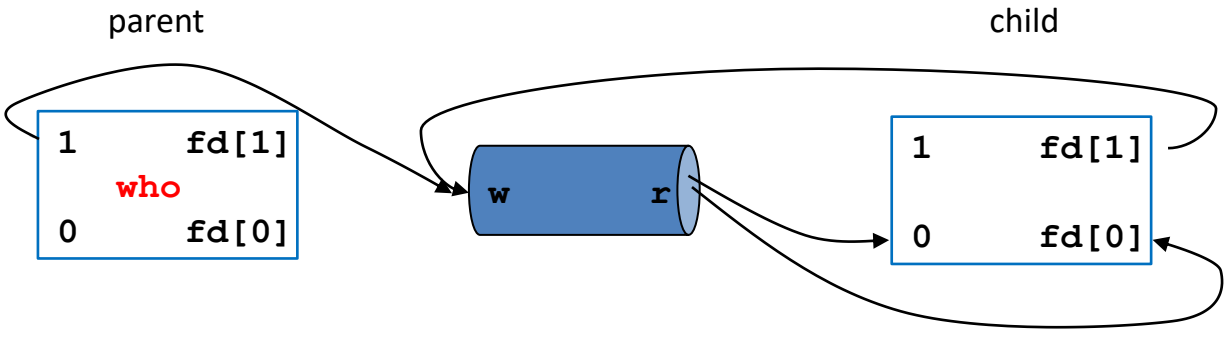
Parent Process

```
execl("/usr/bin/who", "who", (char *) 0);
```



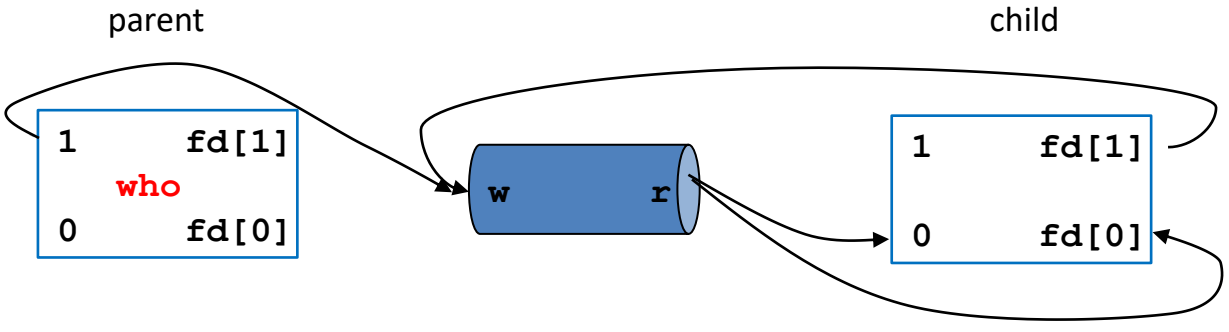
Child Process

```
dup2(fd[0], STDIN_FILENO);
```



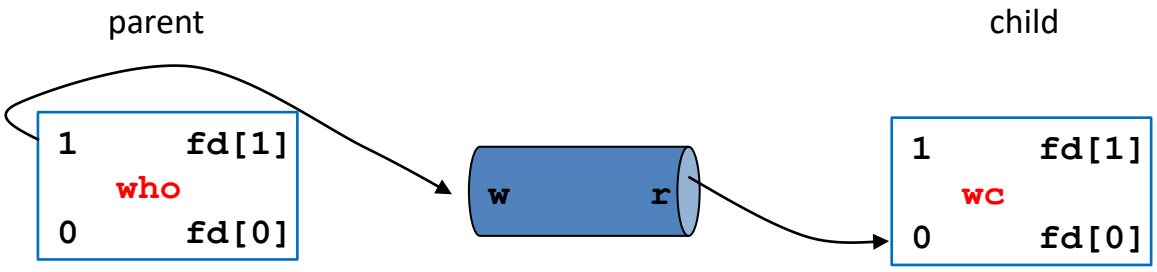
Child Process

```
close(fd[0]);
close(fd[1]);
```



Child Process

```
execl("/usr/bin/wc", "wc", (char *) 0);
```



Putting It All Together

```
if (pipe(fd) == -1) {
        perror("Pipe");
        exit(1);
  }

  switch (fork()) {
  case -1:
        perror("Fork");
        exit(2);
  case 0: /* child */
        dup2(fd[0], STDIN_FILENO);
        close(fd[0]);
        close(fd[1]);
        execl("/usr/bin/wc", "wc", (char *) 0);
        exit(3);
  default: /* parent */
        dup2(fd[1], STDOUT_FILENO);
        close(fd[0]);
        close(fd[1]);
        execl("/usr/bin/who", "who", (char *) 0);
        exit(4);
  }
```